SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: CSA1610 **Student Name:-D.Deepak**
COURSE CODE: Data Warehousing and Data Mining **REG NO:-192572095**

COURSE OUTCOME COVERED

CO5: Apply association rule mining techniques to discover interesting relationships and patterns within large datasets

---

QUESTIONS: 05 **Total Marks:50**

Annexure A

Pseudocode Writing Task (Algorithm Design)

Criteria	Problem Understanding ***(2)***	Algorithm Design ***(3)***	Use of Control Structures ***(2)***	Pseudocode Structure ***(1)***	Completeness ***(2)***	***Total(10)***
Weightage	*20%*	30%	20%	10%	20%	*100%*
Q1						
Q2						
Q3						
Q4						
Q5						

Code of Conduct:

I D .Deepak/192572095(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *deepak*

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Annexure A

Q1. Dataset: Online Shopping Transactions

Order ID	Products Purchased
O1	Mobile, Earphones
O2	Mobile, Charger
O3	Earphones, Charger
O4	Mobile, Earphones, Charger
O5	Mobile, Earphones

Question:
Design pseudocode for the **Apriori algorithm** to discover frequent product combinations from the online shopping dataset using a specified minimum support threshold. Clearly illustrate the candidate itemset generation and pruning process at each iteration.

ANS:-

Apriori Algorithm – Online Shopping Transactions

Given Dataset

Order ID	Products Purchased
O1	Mobile, Earphones
O2	Mobile, Charger
O3	Earphones, Charger
O4	Mobile, Earphones, Charger
O5	Mobile, Earphones

Assume minimum support = 60%.

Total transactions = 5

Therefore,

Minimum support count = 60% × 5 = 3

Pseudocode

Apriori Algorithm for Online Shopping Transactions

Objective

To find frequent product combinations from the online shopping transactions using the Apriori algorithm.

Pseudocode

APRIORI(T, minsup)

Input:

T = set of transactions

minsup = minimum support count

Output:

All frequent itemsets

1. Find all individual items in T.
2. Count the occurrence of every item.
3. Select items whose support count >= minsup.
4. Store them in L1.
5. k = 2

6. Repeat while L(k-1) is not empty:

   a. Generate candidate itemsets Ck

      by joining itemsets in L(k-1).

   b. For every candidate itemset c in Ck:

      Count its occurrence in all transactions.

   c. Remove candidate itemsets whose support

      count is less than minsup.

   d. Store the remaining itemsets in Lk.

   e. k = k + 1

7. Return L1 ∪ L2 ∪ ... ∪ Lk.

Candidate Generation and Pruning

Iteration 1:

Individual items:

- {Mobile} → 4
- {Earphones} → 4
- {Charger} → 3

All satisfy minimum support count 3.

Therefore:

L1 = {{Mobile}, {Earphones}, {Charger}}

Iteration 2 – Candidate Generation:

Generate all possible 2-item combinations:

- {Mobile, Earphones}
- {Mobile, Charger}
- {Earphones, Charger}

Support counts:

- {Mobile, Earphones} → 3
- {Mobile, Charger} → 2

- {Earphones, Charger} → 2

After pruning:

L2 = {{Mobile, Earphones}}

Iteration 3:

There is only one frequent 2-itemset, so a 3-item candidate cannot be generated by joining two different frequent 2-itemsets.

Therefore:

L3 = ∅

Final Frequent Itemsets

{Mobile} → 4/5 = 80%

{Earphones} → 4/5 = 80%

{Charger} → 3/5 = 60%

{Mobile, Earphones} → 3/5 = 60%

Conclusion

Apriori uses the Apriori property: if an itemset is infrequent, all its supersets are also infrequent. Hence, candidate itemsets with insufficient support are pruned at every iteration, reducing unnecessary counting.

Q2. Dataset: Library Book Borrowing Records

Borrow ID	Books Borrowed
B1	Data Science, Python
B2	Python, Machine Learning
B3	Data Science, Machine Learning
B4	Data Science, Python, Machine Learning
B5	Python, Machine Learning

Question:
Develop pseudocode for the **FP-Growth algorithm** to identify frequent borrowing patterns without generating candidate itemsets. Explain the steps involved in constructing the FP-Tree using the given borrowing records. *(BL4)*

ANS:-

Q2. FP-Growth Algorithm for Library Borrowing Records

Objective:-

To identify frequent book-borrowing patterns using FP-Growth without explicitly generating candidate itemsets.

Pseudocode

FP-GROWTH(T, minsup)

Input:

T = set of borrowing transactions

minsup = minimum support count

Output:

Frequent itemsets

1. Scan T and count the support of every item.
2. Remove items having support < minsup.
3. Sort the remaining items in descending order of support.
4. Create an empty FP-Tree.
5. For every transaction t in T:
    a. Remove infrequent items.
    b. Sort remaining items according to the global frequency order.
    c. Insert the ordered items into the FP-Tree.
    d. Increase node counts when the same path already exists.
6. Create a Header Table containing:
    item, support count, and node links.

7. Starting from the least frequent item:

   a. Find its conditional pattern base.

   b. Construct its conditional FP-Tree.

   c. Recursively mine the conditional tree.

8. Combine the discovered patterns with

   the suffix item.

9. Return all frequent itemsets.

Step 1: Count Item Frequencies

Python = 3

Machine Learning = 3

Data Science = 3

All three items satisfy minimum support count 2.

Step 2: Order Items

Since all have equal support, one valid ordering is:

Python → Machine Learning → Data Science

Step 3: Reorder Transactions

B1: Python, Data Science

B2: Python, Machine Learning

B3: Machine Learning, Data Science

B4: Python, Machine Learning, Data Science

B5: Python, Machine Learning

Step 4: Construct FP-Tree

The transactions are inserted one by one.

Common prefixes share the same nodes and their counts are increased.

A simplified representation is:

Root

|

+-- Python:4

| |

| +-- Machine Learning:2

| | |

| | +-- Data Science:1

| |

| +-- Data Science:1

|

+-- Machine Learning:1

  |

  +-- Data Science:1

The exact tree structure depends on the chosen tie-breaking order for equal-frequency items.

Step 5: Mining

The algorithm starts from the least frequent items and constructs conditional pattern bases.

For example, for Machine Learning, the prefix paths containing it are examined to discover combinations such as:

{Python, Machine Learning}

{Machine Learning, Data Science}

The process continues recursively for other items.

Important Point

FP-Growth does not generate candidate itemsets like Apriori. Instead, it compresses the transactions into an FP-Tree and mines frequent patterns directly from the tree.

Conclusion

FP-Growth is efficient because it requires fewer database scans and avoids the costly candidate-generation step used by Apriori. The FP-Tree stores common transaction prefixes, making frequent borrowing patterns easier to discover.

Q3. Dataset: Frequently Purchased Services

Service Combination	Support Count
{Internet}	4
{Streaming}	4
{Cloud Storage}	3
{Internet, Streaming}	3
{Streaming, Cloud Storage}	2

Question:
Write pseudocode to generate **association rules** from the given frequent service combinations. Include the procedures for confidence calculation, rule generation, and filtering based on a minimum confidence threshold. *(BL3)*

ANS:-

Q3. Association Rule Generation

Objective

To generate association rules from frequent service combinations and retain only those rules whose confidence is greater than or equal to the minimum confidence threshold.

Confidence Formula

For a rule:

$X \rightarrow Y$

the confidence is:

$Confidence(X \rightarrow Y)$

$= Support(X \cup Y) / Support(X) \times 100$

Pseudocode

GENERATE-RULES(F, minconf)

Input:

F = set of frequent itemsets

minconf = minimum confidence

Output:

Strong association rules

1. Create an empty set R for rules.

2. For each frequent itemset I in F:

a. If |I| >= 2:

Generate all non-empty proper

subsets of I.

b. For every subset X of I:

Y = I - X

confidence =

support(I) / support(X) × 100

If confidence >= minconf:

Add rule X → Y to R.

3. Return R.

Rule Generation

For {Internet, Streaming}:

Rule 1

Internet → Streaming

Confidence:

= Support(Internet, Streaming) / Support(Internet)

× 100

= 3/4 × 100

= 75%

Since 75% ≥ 70%, the rule is accepted.

Rule 2

Streaming → Internet

Confidence:

= 3/4 × 100

= 75%

This rule is also accepted.

For {Streaming, Cloud Storage}:

Streaming → Cloud Storage

= 2/4 × 100

= 50%

Rejected because 50% < 70%.

Cloud Storage → Streaming

= 2/3 × 100

≈ 66.67%

Rejected because 66.67% < 70%.

Final Strong Rules

Internet → Streaming Confidence = 75%

Streaming → Internet Confidence = 75%

Conclusion

Association rule generation first divides each frequent itemset into an antecedent and consequent. Confidence measures how often the consequent occurs when the antecedent occurs. Rules below the minimum confidence threshold are pruned.

Q4. Dataset: Student Course Enrollment

Student ID	Courses Enrolled
S1	Python, Database
S2	Python, AI
S3	Database, AI
S4	Python, Database, AI
S5	Python

Question:
Design pseudocode for **constraint-based frequent itemset mining** where only itemsets containing the course **"Python"** are considered during the mining process. Explain how the constraint affects candidate generation and pruning. *(BL4)*

ANS:-

Q4. Constraint-Based Frequent Itemset Mining

Objective

To find frequent course combinations subject to the constraint that every itemset must contain the course Python.

Pseudocode

CONSTRAINT-FREQUENT-MINING(T, minsup)

Input:

T = student course transactions

minsup = minimum support count

Constraint:

Every itemset must contain Python.

Output:

Frequent itemsets containing Python

1. Find the support count of Python.

2. If support(Python) < minsup:

    Return empty set.

3. Create L1 = {Python}.

4. Generate candidate itemsets only by adding

   other courses to Python.

5. For every candidate C:

    a. Check whether Python belongs to C.

    b. If Python is not present:

        Prune C immediately.

    c. Count support(C).

    d. If support(C) >= minsup:

        Add C to frequent itemsets.

      Else:

        Prune C.

6. Generate larger candidates only from

   frequent itemsets containing Python.

7. Stop when no new frequent candidates exist.

8. Return all frequent itemsets.

Iteration 1

Support counts:

Python = 4

Database = 3

AI = 2

Therefore:

{Python}

is frequent.

Candidate Generation

Only candidates containing Python are generated:

{Python, Database}

{Python, AI}

The candidate:

{Database, AI}

is not generated, because it does not contain Python.

Support Calculation

{Python, Database}

appears in:

S1, S4

Support count = 2 → Frequent.

{Python, AI}

appears in:

S2, S4

Support count = 2 → Frequent.

Therefore:

L2 =

{Python, Database}

{Python, AI}

Iteration 3

Generate:

{Python, Database, AI}

It appears in:

S4

Support count = 1.

Since:

1 < 2

the candidate is pruned.

Final Frequent Itemsets

{Python} → 4

{Python, Database} → 2

{Python, AI} → 2

Effect of the Constraint

The constraint significantly reduces the search space. Itemsets such as {Database, AI} are discarded immediately because they do not contain Python. Thus, candidate generation and support counting become more efficient.

Conclusion

Constraint-based mining incorporates user-defined conditions directly into the mining process. Here, the Python constraint ensures that only relevant course combinations are generated and evaluated.

Q5. Dataset: Hospital Treatment Hierarchy

Patient ID	Treatments Received
P1	Healthcare, Cardiology, ECG
P2	Healthcare, Orthopedics
P3	Healthcare, Cardiology
P4	Healthcare, Cardiology, ECG
P5	Healthcare, Orthopedics

Question:
Develop pseudocode for **multilevel association rule mining** using the treatment hierarchy provided in the dataset. Explain how rules are generated at different hierarchy levels and how concept hierarchies improve pattern discovery.

ANS:-

Treatment Hierarchy

A simple hierarchy can be represented as:

Healthcare

├── Cardiology

│ └── ECG

└── Orthopedics

Q5. Multilevel Association Rule Mining

Objective

To discover association rules at different levels of the treatment hierarchy, starting from general treatments and moving toward more specific treatments.

Pseudocode

MULTILEVEL-MINING(T, H, minsup)

Input:

T = patient treatment transactions

H = treatment concept hierarchy

minsup = minimum support

Output:

Multilevel frequent itemsets and rules

1. Start at the highest level of the hierarchy.

2. Replace detailed treatments with their

corresponding higher-level concepts.

3. Count the support of items at this level.

4. Keep itemsets satisfying minimum support.

5. Generate association rules from the frequent itemsets.

6. Move to the next lower hierachy level.

7. For every frequent parent concept:

Generate its child concepts.

8. Count the support of the child concepts.

9. Prune concepts whose support is below the level-specific minimum support.

10. Generate rules from the remaining frequent itemsets.

11. Repeat until the lowest hierarchy level is reached.

12. Return rules from all hierarchy levels.

Level 1 – General Level

At the highest level, the main concept is:

Healthcare

It occurs in all five patients.

Therefore:

Support(Healthcare) = 5/5 = 100%

At this level, the pattern is very general.

Level 2 – Treatment Categories

The next level contains:

Cardiology

Orthopedics

Support:

Cardiology = 3

Orthopedics = 2

Thus, both can be considered frequent if the minimum support count is 2.

Level 3 – Specific Treatment

Under Cardiology:

ECG

ECG occurs in:

P1, P4

Therefore:

Support(ECG) = 2

So ECG is also frequent when minimum support count is 2.

Example Rules

At a general level, a rule can be represented as:

Cardiology → Healthcare

Since all Cardiology records are associated with Healthcare:

Confidence = Support(Cardiology, Healthcare)

/ Support(Cardiology)

= 3/3 × 100

= 100%

At a more specific level:

ECG → Cardiology

Since both ECG records belong to Cardiology:

Confidence = 2/2 × 100

= 100%

Multilevel Mining Process

Level 1:

Healthcare

↓

Level 2:

Cardiology, Orthopedics

↓

Level 3:

ECG

The algorithm first discovers broad patterns and then specializes them into more detailed patterns.

Advantages of Concept Hierarchies

1. General patterns can be discovered at higher levels.
2. Specific patterns can be discovered at lower levels.
3. It reduces the complexity of mining detailed data immediately.
4. It helps identify relationships that may not be visible at a single level.
5. It supports analysis at different levels of abstraction.
6. In healthcare data, broad treatment categories can be compared before examining specific procedures.

Conclusion

Multilevel association rule mining uses a concept hierarchy to discover patterns from general to specific levels. In this dataset, the hierarchy allows us to move from **Healthcare → Cardiology/Orthopedics → ECG**, producing useful rules at different levels of detail.